

FORGE — FDE ACADEMY · AI AGENTS ON AZURE · 90 MINUTES

AI Agents on Azure

LangGraph

Multi-Step Reasoning

Tool Use

Nodes & edges · planner-executor · function calling · guard-rails · persistent memory · enterprise framework selection

90-minute session

+ hands-on lab design

Cosmos DB

guard-rails

What we cover in 90 minutes



Foundations & the architecture lens

Agent = deterministic edges + probabilistic core + audited tools

1



LangGraph on Azure

Nodes · edges · state-machine design · checkpointing (Cosmos DB)

2



Single-agent → multi-step reasoning

Planner–executor patterns and re-planning loops

3



Tool use

Function calling · parallel tools · retries · error handling

4



Guard-rails & persistent memory

Where guard-rails go · turn / session / long-term memory

5



Cases, failures & framework choice

2 industry cases · failure scenarios · LangGraph vs the field

6

By the end, you will be able to...



Model an agent as a graph

Design typed state, nodes, and deterministic vs conditional edges in LangGraph.



Decompose reasoning

Split a goal into a planner that thinks and executors that act — inspectably.



Wire tools safely

Add function-calling tools with validation, retries, timeouts and parallelism.



Place guard-rails correctly

Keep the probabilistic core small; audit at edges and tools, not inside the model.



Persist memory on Azure

Use checkpoints for turn/session state and Cosmos DB for long-term recall.



Choose a framework

Justify LangGraph vs Claude SDK, OpenAI Agents SDK and CrewAI for a given use case.

01

FOUNDATIONS

Agents, and the architecture lens



FOUNDATIONS

From LLM to agent



A bare LLM

- Takes text in, predicts text out — one shot.
- Stateless: forgets the previous turn without extra tooling.
- Cannot act on the world — no memory, no tools.
- Great at language; not built for reliable action.

“An LLM is a brain in a jar.



An agent

- Wraps the LLM in a loop with state, tools and memory.
- Stateful: carries context across steps and sessions.
- Reasons, plans, calls tools, observes results, repeats.
- Self-reflects and corrects errors across multiple phases.

*The agent is the brain **plus** the body, senses and notebook.*

MENTAL MODEL

The architecture lens for every agent

agent = deterministic edges + probabilistic core +
audited tools



Deterministic edges

The graph: fixed and conditional routing you can read, test and audit.

Code you trust



Probabilistic core

The LLM calls: planning, judgement and language. Powerful but non-deterministic.

Keep it small & boxed-in



Audited tools

Typed, validated functions to the outside world — each logged and guarded.

Every action is traceable

Design principle: maximise the deterministic shell; minimise and contain the probabilistic core.

FOUNDATIONS

Why enterprises care about this shape



Auditability

Every decision is a state transition you can replay and explain to a regulator.



Control

Human-in-the-loop gates at named nodes for high-impact actions.



Recoverability

Checkpoints let a long job resume instead of restarting after a crash.



Containment

Tools are typed and guarded, so a hallucination can't silently do damage.



Trust is a system-level property — it comes from consistent, inspectable behaviour across time, not from any single clever output.

02

LANGGRAPH ON AZURE

Nodes, edges, state & checkpointing



TERMINOLOGY

LangGraph's four primitives



State

A shared, typed dictionary that flows through every node — the single source of truth. Updated via reducers, never blind overwrites.



Nodes

Python functions that receive state, do work (call the LLM, run a tool, decide), and return a state update patch.



Edges

Transitions between nodes. Normal (always $A \rightarrow B$) or conditional (a router inspects state to choose the next node).



Cycles

Loops are first-class. An agent can call a tool, read the result, reason again — until a stop condition is met.

STATE-MACHINE DESIGN

Typed state + reducers

Why a state machine?

- Defines the allowed states and transitions — invalid combinations are impossible.
- Reducers merge updates safely (e.g. `add_messages` appends, never overwrites).
- Typed objects (`TypedDict` / `Pydantic`) catch shape errors early and aid debugging.
- Determinism: same inputs → same state, so runs are reproducible.

`agent_state.py`

```
from typing import TypedDict, Annotated
from langgraph.graph import add_messages

class AgentState(TypedDict):
    query: str
    plan: list[str]
    # reducer: append, don't overwrite
    msgs: Annotated[list, add_messages]
    confidence: float
    approved: bool
    result: str

# states flow: planning -> executing
#               -> reviewing -> done
```

ROUTING

Deterministic edges vs conditional routers



Normal edge

Always goes A → B. Pure control flow — fully deterministic and testable.



Conditional edge

A router function reads state and returns the name of the next node. The branch keys stay deterministic even when the value driving them came from the LLM.

Decision lives in code; only the input to the decision is probabilistic.

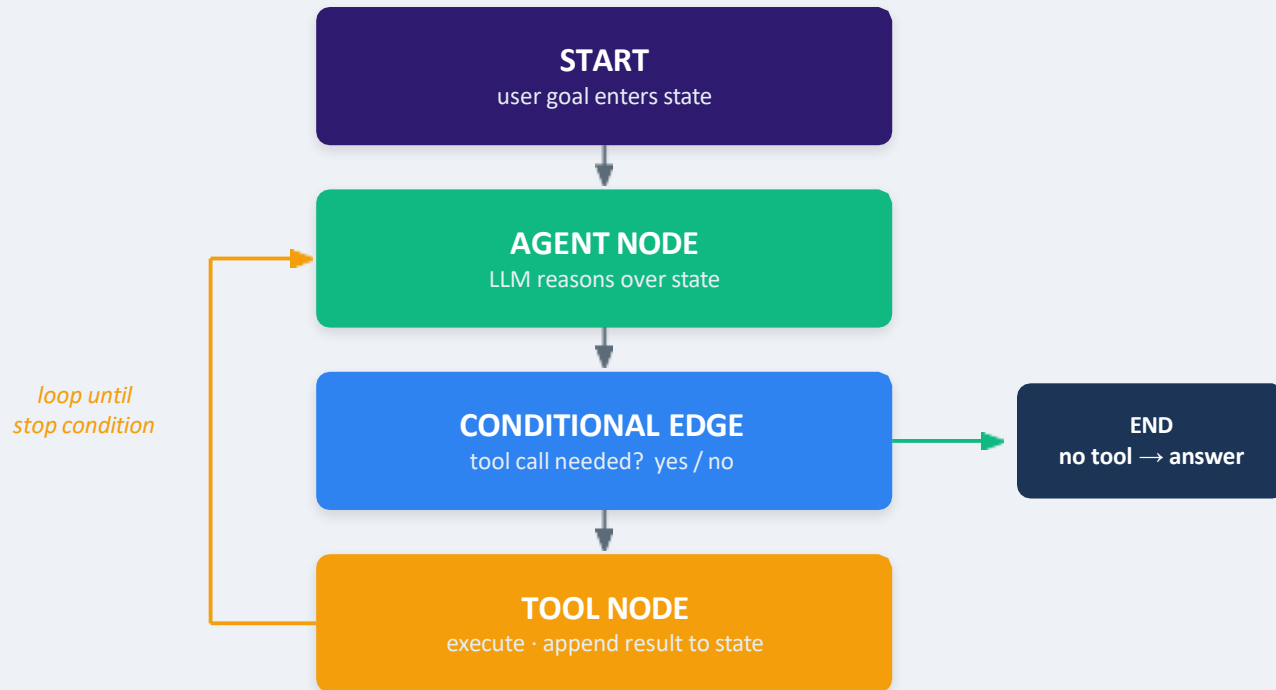
router.py

```
def route(state) -> str:
    # threshold lives in code, not the LLM
    if state['confidence'] >= 0.80:
        return 'auto_execute'
    if state['confidence'] >= 0.50:
        return 'human_review'
    return 'escalate'

graph.add_conditional_edges(
    'assess',
    route,
    {'auto_execute': 'act',
     'human_review': 'review',
     'escalate': 'human'},
)
```

THE AGENT LOOP

Reason → act → observe → repeat (ReAct)



Each pass is one super-step; LangGraph checkpoints state between them, so the loop is resumable.

CHECKPOINTING ON AZURE

Persist & recover with Cosmos DB

What a checkpointer does

- Snapshots full state before/after each node, keyed by thread_id.
- Resume from the last good checkpoint after a crash — no full restart.
- Enables time-travel debugging: list, inspect and replay past steps.
- Backends: in-memory (dev), SQLite, Postgres, Redis — and Azure Cosmos DB (prod).

Cosmos DB = one store for state, chat history, vectors & long-term memory.

azure_checkpoint.py

```
from langchain_azure_cosmosdb \
    import CosmosDBSaverSync

# falls back to Managed Identity
# (DefaultAzureCredential) if no key
checkpointer = CosmosDBSaverSync(
    database_name='agents-db',
    container_name='checkpoints',
    endpoint='<cosmos-endpoint>',
)
app = graph.compile(
    checkpointer=checkpointer)
cfg = {'configurable':
      {'thread_id': 'wo-1042'}}
```

03

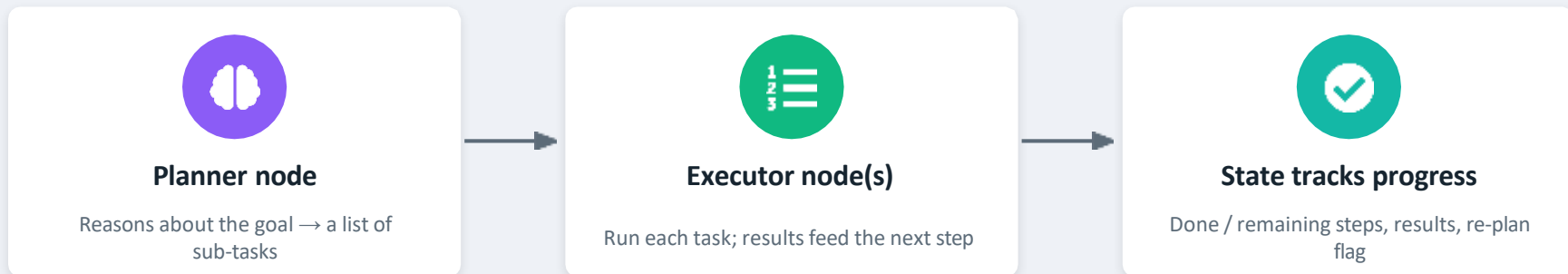
MULTI-STEP REASONING

From single agent to planner–executor



PATTERN

Planner–Executor: separate thinking from doing



Why split them

- Transparency — the plan is inspectable before any action runs.
- Control — a human can edit the plan first.
- Reliability — one task failing doesn't corrupt the whole plan.

`planner_executor.py`

```
def planner(state):
    plan = llm.invoke(
        f"Plan steps for: {state['query']}")
    return {'plan': parse(plan),
            'step': 0}

def executor(state):
    task = state['plan'][state['step']]
    return {'msgs': [run(task)],
            'step': state['step']+1}
```

REASONING LOOPS

Task chains and re-planning

1



Decompose

Planner breaks the goal into atomic, executable steps.

2



Sequence

Tasks run in order; each receives prior results via state.

3



Track

State records completed steps — enabling partial recovery.

4



Branch

Chains fork on intermediate results (conditional routing).

5



Re-plan

New information mid-run can trigger a fresh plan.

A single agent becomes capable of long-horizon work — without losing inspectability — because every step is a recorded state transition.

04

TOOL USE

Function calling, parallelism & resilience



TOOL USE

Function calling: how the model reaches the world

The contract

- You expose a typed schema; the model decides when to call it and with what arguments.
- The framework validates arguments (Pydantic) before the function ever runs.
- The result is appended to state; the model reads it and reasons again.
- The tool is plain code — so its behaviour is deterministic and auditable.

Validate arguments at the boundary — never trust raw LLM output as a function call.

`tool_schema.py`

```
from pydantic import BaseModel, Field

class RaiseWorkOrder(BaseModel):
    asset_id: str
    fault_code: str
    severity: int = Field(ge=1, le=5)
    confidence: float = Field(ge=0, le=1)

@tool(args_schema=RaiseWorkOrder)
def raise_work_order(asset_id, ...):
    """Create a CMMS work order."""
    # typed, validated, logged
    return cmms.create(...)
```

RESILIENCE

Treat the LLM like a flaky vendor

Four verbs that wrap every model and tool call:



Type it

Demand a typed, validated shape on the way in and out.



Validate it

Reject malformed args/outputs before they touch a system.



Retry it

Back-off retries for transient errors and bad JSON.



Time it out

Cap latency so a hung vendor can't stall the graph.



Parallel tools: when the model emits several independent tool calls in one turn, run them concurrently (fan-out), then merge results back into state (fan-in). Cuts latency — but each call still gets its own validate / retry / timeout wrapper.

ERROR HANDLING

Graceful degradation patterns

When this happens...	Do this instead of failing
Tool returns transient error / timeout	Retry with exponential back-off; cap attempts, then fall back.
Model emits malformed / invalid arguments	Re-prompt once with the validation error; else route to human.
Confidence below threshold	Stop auto-acting; escalate to a human reviewer with context.
External system (CMMS/API) unavailable	Queue the action, return a safe partial result, alert ops.
Repeated failure on one branch	Open a circuit-breaker; serve cached/last-known-good state.

Degrade, don't crash: a safe partial answer plus a clear escalation beats a confident wrong action.

05

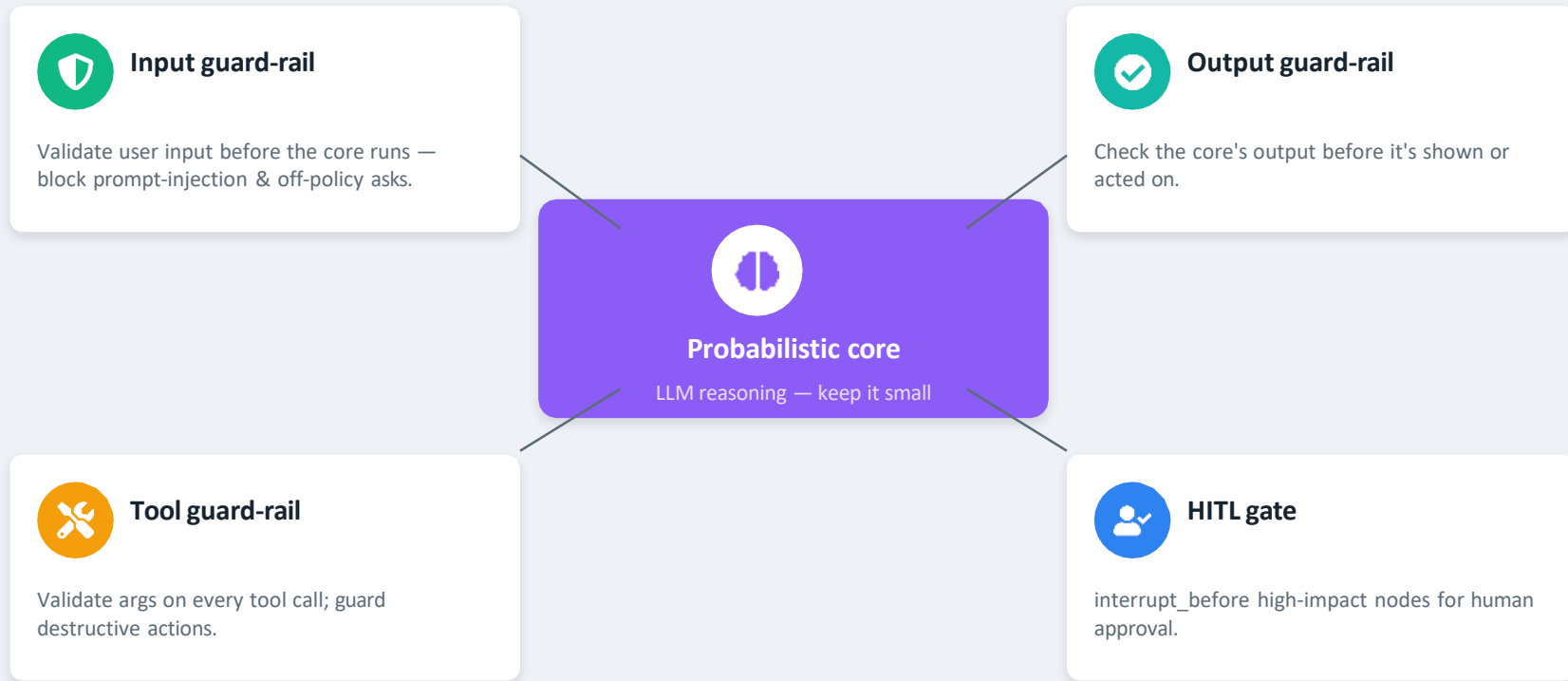
GUARD-RAILS

Deterministic edges vs probabilistic core



PLACEMENT

Guard-rails go around the core, not inside it



You can't make the LLM deterministic — so you wrap it in deterministic checks at every boundary.

REFERENCE

Which guard-rail, where

Guard-rail	Runs	Catches	In LangGraph
Input	Before the first node	Injection, off-policy, PII	Validation node at entry
Output	After the answer node	Unsafe / off-brand output	Validation node before END
Tool	Each tool invocation	Bad args, destructive calls	Wrap tool fn / guard node
HITL	High-impact nodes	Irreversible actions	<code>interrupt_before=[...]</code>
Middleware	Infra-level, all nodes	Policy, audit, rate limits	Decoupled oversight layer

Hard-code human approval as middleware / interrupts — never bury it inside a prompt the model can talk itself out of.

06

PERSISTENT MEMORY

Turn · session · long-term (Cosmos DB)



MEMORY TIERS

Three horizons of memory



TURN

Short-term

The working context inside one agent loop — the messages and intermediate results of the current step.

`Graph state in the loop`



SESSION

Medium-term

Continuity across turns of one conversation/thread. Survives crashes and resumes mid-task.

`Checkpointner · thread_id`



COSMOS DB

Long-term

Facts that outlive the session: user profile, asset history, past resolutions — recalled across threads.

`Store · vector search`

Wider horizon = more durable + slower. Pick the narrowest tier that satisfies the recall you actually need.

IMPLEMENTATION

Memory on Azure: checkpointer + store

Two mechanisms, two jobs

- Checkpointer = infrastructure. Compile once; it persists thread state automatically (short + session).
- Store = product decision. You choose what durable facts to write, how to scope them, when to expire.
- On Azure, Cosmos DB for NoSQL backs both — plus DiskANN vector search for semantic recall.
- Managed Identity (Entra ID) means no secrets in code.

Checkpointer solves session continuity; Store solves relationship continuity.

memory_azure.py

```
from langgraph.store.memory \
    import InMemoryStore
# prod: CosmosDBStore (cross-thread,
#     vector recall via DiskANN)

graph = builder.compile(
    checkpointer=cosmos_saver, # session
    store=long_term_store,     # cross-thread
)

# write a durable fact (long-term)
store.put(('asset', asset_id),
        'last_fault', {'code': 'E14',
                        'fixed_by': 'bearing swap'})

# recalled in a NEW thread later
store.search(('asset', asset_id))
```

07

FAILURE SCENARIOS

How agents fail — and how to catch it



FAILURE MODES

Five ways long-running agents fail



Silent failures

Workflow ends or skips a step with no error — overwritten state, a branch exits early. Hardest to spot.



Cascading errors

A flawed planner output drives wrong executor actions; bad assumptions propagate and amplify downstream.



State drift

State gradually diverges from expected values; equivalent inputs start giving inconsistent outputs.



Recovery without root-cause

Auto-resuming after a crash treats a structural bug as transient — and re-introduces the same fault.



HITL amplification

Rubber-stamp approvals on incomplete context, or oversight that becomes a bottleneck, make things worse.

DESIGN RESPONSE

From failure mode to mitigation

Failure	Mitigation built into the graph
Silent failure	Make partial progress observable; validate state after each node; assert completion signals.
Cascading error	Type the state contract; validate planner output before executors consume it.
State drift	Periodic state-validation checks; ability to recompute critical fields.
Blind recovery	Resume only after root-cause check; context-aware recovery paths, not auto-retry.
HITL amplification	Surface explanations at decision points; show intermediate results before irreversible actions.

Treat state as a controlled interface, not unrestricted memory — and test against simulated crashes.

08

CASE STUDY 1 · LAB

Machine-maintenance agent



INDUSTRY CASE 1

Agentic predictive maintenance

The scenario

- An agent continuously reads machine-maintenance logs and sensor telemetry from a plant.
- It detects a deviation, diagnoses the likely fault against logs + manuals, and drafts a work order.
- A work-order agent pre-fills the CMMS ticket — estimated time, parts, required skills.
- Crucially: a human-in-the-loop checkpoint lets a supervisor review/edit before it is finalised.
- When the agent's confidence is low, it does NOT auto-act — it escalates to a human.

~30 sec

Agentic workflows compress a coordination chain that traditionally spans 3–4 people and up to two weeks into seconds — querying CMMS, ERP and the production calendar automatically.

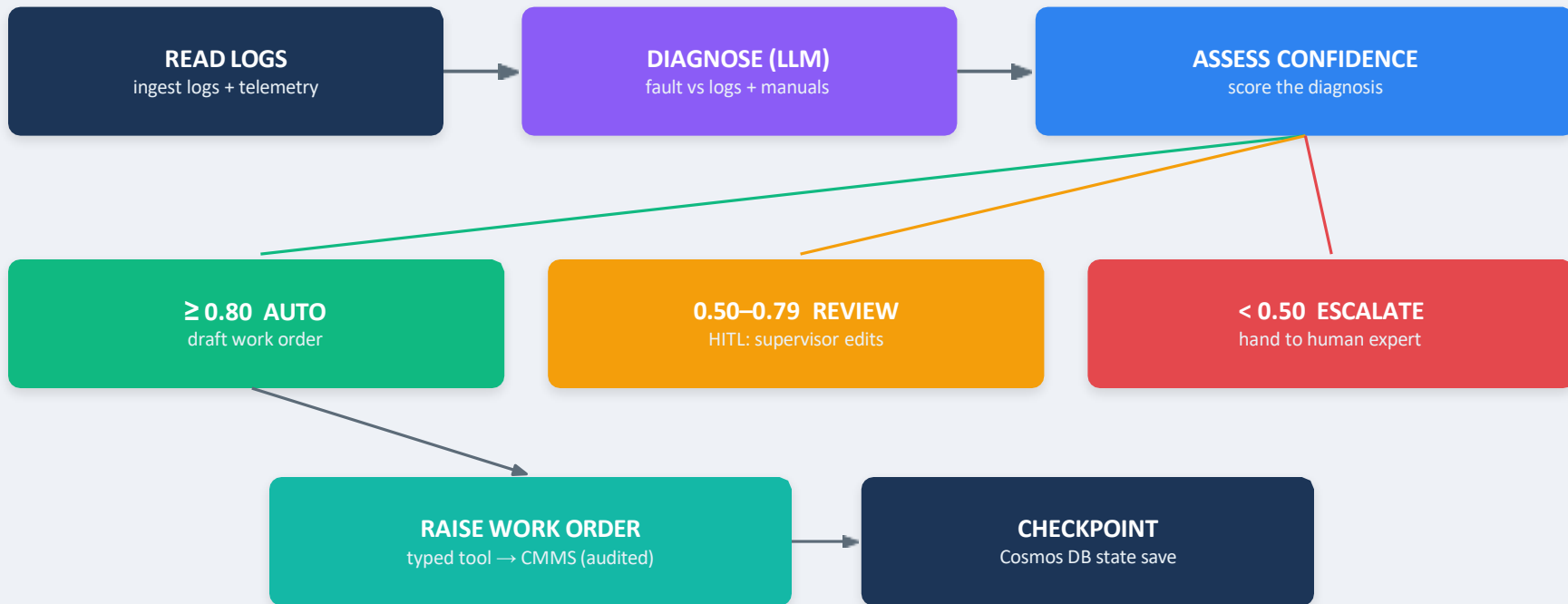


References

[IIoT World — agentic maintenance panel \(TDK/AWS/Omron/Acerta\)](#)
[The AI Journal — multi-agent maintenance + HITL work-order agent](#)
[GM-style CMMS case — alerts with probability + recommended action](#)

LAB · GRAPH DESIGN

The maintenance agent as a LangGraph



Green = deterministic edges · Purple = probabilistic core · Teal = audited tool. The confidence threshold is plain code on a conditional edge.

LAB · RESILIENCE FLOW

Tool-use + graceful degradation

Confidence-gated action

- The diagnose node returns a structured result with a confidence field.
- A deterministic router maps confidence → auto / review / escalate.
- `raise_work_order` is a typed tool: validated args, retries, timeout, full audit log.
- If the CMMS is down: queue the order, return a safe partial result, alert ops — never silently drop it.
- Low confidence is treated as a first-class outcome, not an error.

`degradation.py`

```
def assess(state):
    c = state['confidence']
    if c >= 0.80:  nxt = 'auto'
    elif c >= 0.50: nxt = 'review'
    else:          nxt = 'escalate'
    return {'route': nxt}

@retry(max=3, backoff='expo')
@timeout(seconds=10)
def raise_work_order(wo: WorkOrder):
    try:
        return cmms.create(wo)
    except CmmsDown:
        queue.put(wo)          # degrade
        alert_ops('cmms down')
        return Partial(wo, 'queued')
```

LAB · BUILD SEQUENCE

What learners build, step by step

1

Define typed AgentState

query, diagnosis, confidence, route, approved, result — with reducers.

2

Add nodes

read_logs → diagnose (LLM) → assess → act / review / escalate.

3

Wire the conditional edge

Router on the confidence threshold (deterministic).

4

Implement the typed tool

raise_work_order with Pydantic schema, retries and timeout.

5

Add the HITL gate

interrupt_before the auto-execute node for the review band.

6

Compile with a checkpointer

Cosmos DB saver keyed by work-order thread_id; break it & resume.

09

CASE STUDY 2

When an agent fails in production



INDUSTRY CASE 2

Air Canada's chatbot — a costly failure

What happened

- A grieving customer asked Air Canada's website chatbot about bereavement fares.
- The bot confidently stated he could claim the discount retroactively — which contradicted the airline's actual policy.
- Relying on it, he booked full fare; the refund was then denied.
- A tribunal held the airline liable for negligent misrepresentation — the chatbot is part of its website, not a separate entity.
- Air Canada was ordered to pay damages.



Three reliability failures

- Wrong answer delivered with high apparent confidence.
- No uncertainty signal flagged it as unreliable.
- No mechanism to defer policy questions to an authoritative source.

[Reference: BC Civil Resolution Tribunal — Moffatt v. Air Canada \(2024\)](#)

LESSON

How the architecture lens prevents it



Ground in source

Policy answers come from a retrieved, authoritative document — not free-form generation. RAG with citation, output guard-rail blocks ungrounded claims.



Carry a confidence signal

The model returns a confidence/uncertainty field; low confidence routes away from auto-answering.



Defer & escalate

For high-liability topics, a conditional edge hands off to a human or a verified policy lookup instead of guessing.



Audit every answer

Each response is a logged state transition — so 'what did the bot say and why' is always answerable.

The failure wasn't the LLM being wrong — it was the absence of deterministic edges, guard-rails and an escalation path around it.

10

FRAMEWORK CHOICE

LangGraph vs Claude SDK vs OpenAI SDK vs CrewAI



THE FIELD

Four ways to build an agent



LangGraph

Maximum control

Stateful graph orchestration. You build it like a state machine — explicit nodes, edges, typed state, checkpoints.



Claude Agent SDK

Autonomous power

Give Claude a computer and a goal. Built-in tools (read/edit/bash/web), autonomous loop, session store.



OpenAI Agents SDK

Fast & simple

Lightweight: four primitives — Agents, Tools, Handoffs, Guardrails. Minimal abstraction, built-in tracing.



CrewAI

Team intelligence

Think in roles, not nodes. Agents as specialists with per-agent memory; YAML config for non-developers.

SIDE-BY-SIDE

How the four compare

	LangGraph	Claude SDK	OpenAI SDK	CrewAI
Abstraction	Low — explicit graph	Highest — write a prompt	Low/medium — 4 primitives	High — roles + tasks
Control	Full, fine-grained	Autonomous (less)	Medium	Structured
State	Typed dict + checkpoints	Session store (S3/Redis)	Sessions layer	Per-agent memory
Tools	You wire the loop	Built-in, autonomous	Function tools + hosted	Framework-managed
Multi-agent	Supervisor/Swarm/Subgraph	Subagents via MCP	Handoffs	Sequential/Hierarchical
LLM support	Any (via LangChain)	Claude only	OpenAI-first, ~any compat	Any (per-agent swap)
Best fit	Complex audited pipelines	Autonomous task execution	Rapid app-team agents	Business team workflows

CrewAI reports ~5.76x faster setup than LangGraph on benchmark tasks; LangGraph trades that for control and auditability.

THE VERDICT

Why LangGraph leads in the enterprise



Auditability by design

Control flow and state changes are explicit — replayable traces satisfy regulators in banking, healthcare and legal.



Native human-in-the-loop

`interrupt_before` / `interrupt_after` at any node — approval gates without hacking the prompt.



Checkpoint & recover

Long-running, multi-branch jobs resume after failure; time-travel debugging finds root causes.



Deterministic control

Complex conditional branching and cycles live in tested code, not model whim.



Azure-native persistence

First-class Cosmos DB checkpointer + store + vector search; Managed Identity, 99.999% SLA.



Composable & model-agnostic

Any LLM via LangChain; subgraphs and even other frameworks (CrewAI/Claude SDK) as nodes.

DECISION GUIDE

Best-fit use cases per framework

LangGraph

- Loan decisioning with a human review node
- Clinical triage across specialist handoffs
- Document review: escalate / approve / reject
- Threat-detection loop: scan→analyse→patch→verify
- Predictive-maintenance agent (our lab)

CrewAI

- Researcher→Writer→Editor→Fact-checker crew
- Analyst+Quant+Risk investment reports
- Support triage→specialist→escalation
- CV-screening crew mirroring a hiring team

Claude / OpenAI SDK

- Autonomous bug-fix agent (read/edit/test/PR)
- Deep-research across many sources & sessions
- IT-ops agent: logs→diagnose→patch→report
- Rapid app-team agents with handoffs (OpenAI)

***Use them together:** LangGraph can orchestrate CrewAI crews or Claude/OpenAI SDK tasks as nodes — the strongest systems combine control, autonomy and role-based teams.*

11

REFERENCE

Glossary of terms & jargon



GLOSSARY

Agents, graphs & reasoning

Agent

An LLM wrapped in a loop with state, tools and memory so it can reason, act and self-correct.

LangGraph

A framework that models agent workflows as directed graphs of nodes and edges over typed state.

StateGraph

The LangGraph object that defines a graph-based workflow.

Node

A Python function that reads state, does work, and returns a state-update patch.

Edge

A transition between nodes — normal (fixed $A \rightarrow B$) or conditional (a router chooses).

State / typed state

The shared dictionary flowing through the graph, defined with TypedDict or Pydantic.

Reducer

A function controlling how a state field updates — e.g. `add_messages` appends instead of overwriting.

Cycle

A loop in the graph; lets an agent reason $\rightarrow$ act $\rightarrow$ observe repeatedly until a stop condition.

ReAct loop

Reason-and-act pattern: the agent alternates LLM reasoning with tool calls.

Planner–Executor

Pattern separating a planner (decomposes the goal) from executors (run each step).

Conditional router

A function that inspects state and returns the name of the next node to run.

Supervisor–Worker

Multi-agent topology where a supervisor routes work to specialist worker agents.

GLOSSARY

Tools, memory & operations

Function calling

The model emits a structured request to run a typed tool with validated arguments.

Tool

A typed, validated function the agent can call to act on the outside world (DB, API, file).

Parallel tools

Several independent tool calls in one turn run concurrently (fan-out), merged back (fan-in).

Guard-rail

A deterministic check on input, output or a tool call that can halt unsafe behaviour.

HITL

Human-in-the-loop: pausing at a node (interrupt_before) for a human to approve/edit/reject.

Checkpoint

Persists thread state per super-step; enables resume, recovery and time-travel

thread_id

Key that isolates one run's checkpoint namespace, so sessions don't collide.

Time-travel debugging

Replaying or inspecting a past checkpoint to see exactly where a run diverged.

Azure Cosmos DB

Globally-distributed Azure database backing state, chat history, vectors and memory.

DiskANN

Cosmos DB vector index for low-latency similarity search at scale.

Graceful degradation

Returning a safe partial result and escalating instead of crashing on failure.

Circuit-breaker

Stops calling a repeatedly-failing dependency; serves a fallback until it recovers.

GO DEEPER

References & further reading



LangGraph persistence (checkpointers & stores)

docs.langchain.com/oss/python/langgraph/persistence



LangChain × Azure Cosmos DB — agents & RAG in one DB

devblogs.microsoft.com/cosmosdb



Sample: multi-agent LangGraph on Azure Cosmos DB

github.com/AzureCosmosDB/multi-agent-langgraph



OpenAI Agents SDK (agents, tools, handoffs, guardrails)

openai.github.io/openai-agents-python



Claude Agent SDK

platform.claude.com/docs/agent-sdk



CrewAI documentation

docs.crewai.com



Case 1 — Agentic maintenance (IIoT World panel)

iiot-world.com · [agentic-ai manufacturing maintenance](#)



Case 2 — Moffatt v. Air Canada chatbot liability

[Forbes](#) · [BC Civil Resolution Tribunal \(2024\)](#)

RECAP

Build the shell, box the core, audit the tools

- LangGraph models agents as typed state + deterministic edges — readable, testable, auditable.
- Checkpointers and Cosmos DB give resumable runs and three tiers of persistent memory.
- Tools are typed, validated, retried and timed-out; low confidence escalates, it doesn't guess.
- Guard-rails live at the boundaries; the probabilistic core stays small and contained.
- LangGraph wins enterprise work on control, audit, HITL and Azure-native persistence.